



5

USER ENVIRONMENT AND INTERACTION DETECTION



As automated malware sandboxes get better at hiding themselves from evasive malware, malware authors must adapt. One tactic they use is to enumerate the user's environment and the user's interaction with it. As [Chapter 4](#) noted, the everyday user's setup has open browser tabs, many windows open and apps in use, and frequent mouse and keyboard interaction, making it quite different from the sandbox environment. An automated malware analysis sandbox is designed to boot up, detonate a malware sample, and then promptly shut down. It may not exhibit any normal user behaviors or other indicators that suggest it's a valid end-user system.

Modern malware can look for evidence of a real user by searching for typical user behaviors, such as downloaded browser cookies and desktop wallpaper settings or mouse and keyboard interactions. In this

chapter, I'll outline some interesting techniques that malware uses to accomplish this.

Browser Cookies, Cache, and Browsing History

Some malware may be able to enumerate the host's internet cookies, cache, and browsing history. *Cookies* are small files that web pages save to the disk, usually for storing the user's website configuration and preferences. Depending on the browser and version, cookies can be stored in individual files or in a small database, such as SQLite. The *cache* is a file or group of files that stores website resources such as images so that the page loads more quickly the next time the user visits it. Much like cookies, the cache can be stored in multiple files or in a database. Finally, the *browsing history* is simply a list of previously visited websites, typically stored as one or more database files.

The typical end user will likely have hundreds or thousands of stored cookie and cache files and a large internet browsing history, whereas a typical sandbox or malware analysis system may not have any at all. Malware can take advantage of this discrepancy by counting the number of cookies, cache entries, or previously visited websites and checking it against a threshold value. For example, if the victim machine has only five entries in its browsing history, the malware might assume it's running in a clean sandbox environment.

Every browser has standard locations for cookies, cache files, and browsing history that malware might attempt to enumerate. Here are some of the most common:

Chrome

- *C:\Users\<user>\AppData\Local\Google\Chrome\User Data\Default*
- *C:\Users\<user>\AppData\Local\Google\Chrome\User Data\Default\Cache*

- *C:\Users\<user>\AppData\Local\Google\Chrome\User Data\Default\History*

Firefox

- *C:\Users\<user>\AppData\Local\Mozilla\Firefox\Profiles*
- *C:\Users\<user>\AppData\Roaming\Mozilla\Firefox\Profiles*

Internet Explorer

- *C:\Users\<user>\AppData\Roaming\Microsoft\Windows\Cookies*
- *C:\Users\<user>\AppData\Local\Microsoft\Windows\Temporary Internet Files*
- *C:\Users\<user>\AppData\Local\Microsoft\Windows\WebCache*
- *C:\Users\<user>\AppData\Local\Microsoft\Internet Explorer\Recovery*

Edge

- *C:\Users\<user>\AppData\Local\Packages\<package name>\AC\MicrosoftEdge\User*
- *C:\Users\<user>\AppData\Local\Packages\<package name>\AC\MicrosoftEdge\Cache*

This list is non-exhaustive, and locations may change, of course, depending on the Windows OS and browser versions in use.

If you spot malware enumerating through these files (possibly by calling Windows functions such as `FindFirstFile` and `FindNextFile`), it may be attempting to detect the analysis environment. Malware may also use `FindFirstUrlCacheEntry` and `FindNextUrlCacheEntry`, which sequentially enumerate browser cache entries. These APIs are specific to Microsoft browser caches, however. Once again, the enumeration method will largely depend on the browser and version being used.

Older browsers and versions typically use multiple small files for cookies, cache, and history storage, while modern browsers use data-

bases. If the browser cookies, cache, and history are stored in database files, the malware may attempt to interact with them directly. For example, in either the malware executable file or its process memory address space, you may spot static strings that reference certain browser directories (such as `C:\Users\<user>\AppData\Local\Google\Chrome\User Data\Default\History`), followed by a database query such as this:

```
SELECT title FROM urls
```

This command could be used to enumerate all of the web history in the history database. Database interaction is beyond the scope of this book, so we won't go into further detail here, but it's important to be aware of this technique.

Recent Office Files

Using recent Office files is another good way for malware to determine whether it's running in an analysis lab. A real end user will likely have opened many files with Microsoft Office applications, and Windows keeps track of those files. When you open a document in Word, for example, the file will be added to your *Office Recent Files* list.

Information about your recent Office files is contained in the registry key `HKEY_CURRENT_USER\SOFTWARE\Microsoft\Office\<Office_Version_Number>`, under a specific subkey called `Excel`, `Word`, `Powerpoint`, and so on. Further information may be stored in the filesystem directory `C:\Users\<user>\AppData\Roaming\Microsoft\Office\Recent`. If you spot malware enumerating this registry key or folder path (using any of the previously mentioned Windows functions for file and registry enumeration), it may very well be attempting to identify recent Office documents to determine whether the victim host is being used by a “real” end user.

User Files and Directories

A typical user will probably have many files on the system in various user directories, such as *Documents*, *Pictures*, *Desktop*, and so on.

Using the file enumeration methods described in [Chapter 4](#), malware can enumerate these directories in order to sense whether the host is a real user or not. If the malware discovers a lack of user activity in these directories, it might conclude that it's running in a sandbox or analysis environment and take evasive actions.

Desktop Wallpaper

One particularly creative method malware uses to detect analysis machines is checking the currently configured wallpaper, since authentic users tend to change their desktop wallpaper from the Windows default. To do so, the malware can simply check the Wallpaper registry value `HKEY_CURRENT_USER\Software\Microsoft\Internet Explorer\Desktop\General\WallpaperSource`. If the user is still using the default Windows wallpaper, the `WallpaperSource` value will contain that wallpaper's path, which is somewhere in the `C:\Windows\` directory. On the other hand, if the user has custom desktop wallpaper configured, the `WallpaperSource` value will likely contain a custom directory and image name, such as `C:\Users\<user>\Pictures\my_wallpaper.jpg`.

Desktop Windows

Some malware variants count the number of active desktop windows or search for specific ones. They can use the function `GetForegroundWindow` to test whether the foreground window (meaning the currently active window) changes. Since I'm typing this text in LibreOffice, this program is my active foreground window. As a legitimate user, my active window will likely change quite a bit; for example, I might minimize LibreOffice to take a break from writing and watch YouTube cat videos in Chrome. In an automated malware analysis sandbox environment, the active window probably won't change

much. Some malware variants know this and can use it against the analysis system for detection. In this example, the malware is checking whether the foreground window has changed after five seconds:

```
loc_34E642:  
call GetForegroundWindow  
mov dword ptr ds:[ebx+WindowHandle], eax  
push 1388h ; "5s"  
call Sleep  
call GetForegroundWindow  
cmp dword ptr ds:[ebx+WindowHandle], eax  
je loc_34E642
```

First, the malware calls `GetForegroundWindow`, which returns a handle to the current foreground window and stores it in the buffer at address `[ebx+WindowHandle]`. Next, the malware calls the `sleep` function, which will pause the sample for five seconds. The malware makes a second call to `GetForegroundWindow` and then compares the handle values of the two `GetForegroundWindow` calls with `cmp dword ptr ds:[ebx+WindowHandle], eax`. If the handles match (meaning that the foreground window hasn't changed), this routine loops over again. This malware sample could continue to loop indefinitely, possibly avoiding analysis in an automated sandbox completely, or it could loop several times and then terminate itself. Either scenario presents an interesting challenge for a malware analysis sandbox. Fortunately, many modern sandboxes simulate user activity to thwart this technique. Some can even run in interactive mode, which allows you to directly interact with the malware inside the sandbox, also helping to circumvent this type of tactic.

Alternatively, a malware sample can use the `EnumWindows` function, which returns the number of open windows on the user's desktop. Windows creates many window objects for various reasons, so in a normal user environment, this number will be fairly high. For example, I ran `EnumWindows` on my personal system and it returned a value of 97 windows! In a sandbox analysis environment, this number will

likely be substantially lower. This code snippet demonstrates the use of the `EnumWindows` function:

```
push ebx
call EnumWindows
pop eax
cmp eax, 20
jle terminate_process
```

`EnumWindows` takes a parameter that essentially serves as a pointer to a buffer that will store the result of the function call (`push ebx`). After `EnumWindows` is called, the `pop eax` instruction will pop the pointer from the `ebx` buffer off the stack and into `eax`. The malware compares the `EnumWindows` value (now stored in `eax`) to 20, and if the number of open windows is less than or equal to this value, the sample will terminate itself. This sample is assuming that a malware analysis sandbox will have 20 or fewer windows activated at one time.

Besides enumerating the number of active windows on the victim system or sensing whether the foreground window is changing, malware can also actively search for a specific application window. This is useful for two reasons. First, the malware sample can search for open windows running applications that are commonly run by typical end users: Microsoft Office products, email programs, browsers, and so on. If enough of these applications are open, the malware could reasonably assume it's *not* running in a malware analyst's lab. Second, the malware sample can look for certain malware analysis tools, similar to what I described in "Processes" in [Chapter 4](#). For example, the malware might search for open windows that contain the term *Procmon*, *Fiddler*, or *Wireshark*, typically by calling the function `EnumWindows` or `FindWindow`. As with looking for a certain process, it iterates through open windows and compares the title of each to a string, and the result can clue it in to the fact that it's being analyzed.

Mouse and Keyboard Interaction

Everyday end users are almost always using their mouse to move their cursor around the screen, whether they're browsing the internet, editing a document, or playing a video game. Some malware can detect this activity using `GetCursorPos`, which returns the coordinates of the user's mouse cursor. The following pseudocode shows what this might look like in action:

```
GetCursorPos(&CursorPos1)
Sleep(30)
GetCursorPos(&CursorPos2)

if (CursorPos1 == CursorPos2)
    TerminateProcess()
```

First, the malware calls the `GetCursorPos` function and stores the resulting mouse cursor coordinates in the `CursorPos1` buffer. Next, it calls the `sleep` function, which pauses the malware's execution for 30 seconds, and then it calls `GetCursorPos` again. Finally, it compares the two resulting cursor coordinate values, and if they're the same (meaning the cursor hasn't moved), the sample will terminate itself. You can probably see how this is an effective method for evading automated sandboxes, as the cursor is unlikely to move itself (unless, of course, the sandbox is designed to mimic a real user).

Another similar technique involves the malware waiting for certain mouse buttons to be pressed or a certain number of mouse clicks to occur before it executes its malicious code. FireEye wrote a 2012 research article, "Hot Knives Through Butter: Evading File-based Sandboxes," about this particular technique being used by a malware family called Upclicker. To monitor these mouse actions, Upclicker established a *hook* on the mouse, allowing the malware to intercept and monitor all mouse activity and wait for certain events to occur. Here's what this might look like in malware code:


```
push offset jump_location
push 0Eh
call SetWindowsHookExA
--snip--
loc jump_location:
call do_evil_things
```

The malware sample first pushes the `jump_location` parameter to the stack; this is where the malware will jump to when a certain mouse event occurs. Another parameter, `0Eh` in hexadecimal (or 14 in decimal), tells `SetWindowsHookExA` to hook mouse actions. The call to `SetWindowsHookExA` tells the program to jump to the code specified in `jump_location` once the victim user clicks a mouse button.

This code is simplified for the sake of brevity. In reality, the malware would likely implement extra logic to take action only upon certain mouse events, such as a left-button click (as in the case of Upclicker). To read more about Upclicker and also get a good introduction to sandbox evasion, check out the FireEye report at <https://media.blackhat.com/us-13/US-13-Singh-Hot-Knives-Through-Butter-Evading-File-based-Sandboxes-WP.pdf>.

This hooking magic doesn't just work for the mouse. Malware can also hook the keyboard by passing `0Dh` (13 in decimal) to the `SetWindowsHookEx` function instead, then waiting for a certain key to be pressed before fully executing. (Hooking will be discussed in more detail in [Chapters 8](#) and [12](#).) Alternatively, malware could also call the function `GetAsyncKeyState` to monitor for keypresses.

Monitoring mouse and keyboard interactions can be a very effective method of detecting and bypassing an automated malware analysis sandbox. Unless the sandbox or malware analyst presses the specific keys or mouse buttons, the malware sample may look completely benign in the context of the sandbox environment.

NOTE

To simulate a real end-user environment, make your analysis VMs and sandboxes look as much like a real user as possible. Changing your wallpaper and visiting some websites (to populate your cookies and cache directories) can go a long way. Even opening additional windows and moving the mouse around the screen may help avoid some of these detection techniques, even if you feel a bit silly doing it.

System Uptime

System uptime is the length of time the system has been powered on, and it can be a great indicator to malware that it's in an analysis environment. A typical end-user device will likely be powered on for hours, if not days, at a time. Servers may be powered on for months or years without a reboot. Since malware analysts typically boot up their VMs and sandboxes to analyze a malware sample on demand, a short system uptime can be a big hint that the system is an analysis machine.

There are multiple ways to check for system uptime, via both the Windows API and other helper commands. Perhaps the most common method is the `GetTickCount` Windows API function, which returns the system uptime in milliseconds. A *tick* is created by the processor clock, which is responsible for keeping time and coordinating instructions. When a system is shut down or rebooted, `GetTickCount` essentially resets to 0. The following code uses `GetTickCount` to see if the system has been powered on for 20 minutes:

```
mov ebx, 124F80h
call GetTickCount
cmp eax, ebx
jb  terminate_process
```

The sample first moves `124F80` in hex (`1200000` in decimal) into the `ebx` register, representing 1,200,000 milliseconds, or 20 minutes. Then, it calls `GetTickCount` and compares the returned tick count value to the value in `ebx`. If the value from `GetTickCount` is below the `ebx` value, meaning the system has been powered on for less than 20 minutes, the malware sample terminates itself.

Malware may also use the Windows command line to get the system uptime. Options include the `sysinfo` command, which returns a list of information about the system, including the uptime; `uptime.exe`, a binary included with most versions of Windows; and the `net statistics workstation` command. Finally, malware can invoke WMIC to return the system uptime using the command `wmic os get lastbootuptime`.

One final important note here is that `GetTickCount` and the other methods mentioned are often used in both benign and malicious applications, and not just for exposing analysis environments and sandboxes. Just because a malware sample is inspecting the system uptime doesn't mean it's acting evasively, but you should treat the behavior as a red flag.

Summary

In this chapter, we covered some creative and sneaky ways in which malware can enumerate the environment and look for evidence of actual user activity. You can thwart many of these user detection techniques simply by designing your analysis environment to make it look legitimate to malware. Some of these changes, such as changing the default Windows wallpaper and ensuring you have some items in your browsing history, are simple to implement. We'll discuss other ways to thwart detection techniques in [Appendix A](#). Note also that some advanced sandboxes have built-in protections against many of these techniques.

In the next chapter, we'll look at how evasive malware can enumerate system hardware and network device information to detect a VM analysis environment.